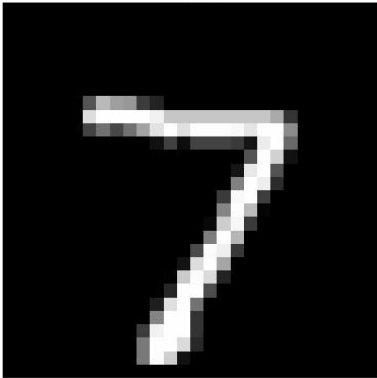




NAMA :	MUHAMMAD NABIRULLAH BIN ZUKAIRI
KELAS :	1 DKA SEM 2
TARIKH	27/8/2026
TAJUK :	LAPORAN AMALI 1
NAMA PENSYARAH :	SARASWATHY A/P CHANDIRAN

LANGKAH	JAWAPAN
1. LANGKAH 1 - Sediakan folder projek dan Git	Langkah 1 — Sediakan Folder Projek & Git - Cipta folder dengan nama : DKA3223_AMALI1_NABIL - Cipta repository di GitHub
2. LANGKAH 2 - Import library Import library yang diperlukan untuk tensor, neural network, dataset, DataLoader dan paparan imej. Contoh output semakan: <pre>PyTorch version: 2.x.x Device: cpu</pre>	<pre>import torch import torch.nn as nn import torch.optim as optim from torch.utils.data import DataLoader from torchvision import datasets, transforms import matplotlib.pyplot as plt # Semak versi PyTorch dan peranti yang digunakan (CPU/GPU) device = torch.device("cuda" if torch.cuda.is_available() else "cpu") print("PyTorch version:", torch.__version__) print("Device:", device)</pre> <pre>... PyTorch version: 2.11.0+cpu Device: cpu</pre>
3. LANGKAH 3 - Muatkan dan semak dataset MNIST Contoh output semakan: <pre>Training data: 60000 Testing data: 10000 Image shape: torch.Size([1, 28, 28]) Actual label: 7 [Paparan satu imej digit MNIST]</pre>	<pre>... Training data: 60000 Testing data: 10000 Image shape: torch.Size([1, 28, 28]) Actual label: 7</pre> Label: 7 
4. LANGKAH 4 - Bina model ANN Contoh bentuk output: <pre>Flatten(start_dim=1) Linear(in_features=784, out_features=...) ReLU() Linear(in_features=..., out_features=10)</pre>	<pre>class ANN(nn.Module): def __init__(self): super(ANN, self).__init__() self.flatten = nn.Flatten() self.fc1 = nn.Linear(28*28, 128) # input 784 -> hidden layer 128 self.relu = nn.ReLU() self.fc2 = nn.Linear(128, 10) # hidden layer -> output 10 kelas def forward(self, x): x = self.flatten(x) x = self.fc1(x) x = self.relu(x) x = self.fc2(x) return x ann_model = ANN().to(device) print(ann_model)</pre> <pre># 3.4.4 - Loss function & optimizer criterion_ann = nn.CrossEntropyLoss() optimizer_ann = optim.Adam(ann_model.parameters(), lr=0.001)</pre> <pre>... ANN((flatten): Flatten(start_dim=1, end_dim=-1) (fc1): Linear(in_features=784, out_features=128, bias=True) (relu): ReLU() (fc2): Linear(in_features=128, out_features=10, bias=True))</pre>

5. LANGKAH 5 - Latih dan uji ANN

Contoh output semakan (nilai sebenar boleh berbeza):

```
Epoch 1/2 - Loss: 0.xxxx  
Epoch 2/2 - Loss: 0.xxxx  
ANN Testing Accuracy: xx.xx%
```

```
for images, labels in train_loader:  
    images, labels = images.to(device), labels.to(device)  
  
    optimizer_ann.zero_grad()  
    outputs = ann_model(images)  
    loss = criterion_ann(outputs, labels)  
    loss.backward()  
    optimizer_ann.step()  
  
    running_loss += loss.item()  
  
# 3.5.2 - Papar purata loss selepas setiap epoch  
avg_loss = running_loss / len(train_loader)  
print(f"Epoch {epoch+1}/{num_epochs} - Loss: {avg_loss:.4f}")
```

```
Epoch 1/2 - Loss: 0.3461  
Epoch 2/2 - Loss: 0.1581
```

```
▶ # 3.5.3 - Tukar ke evaluation mode & kira accuracy  
ann_model.eval()  
correct = 0  
total = 0  
  
with torch.no_grad():  
    for images, labels in test_loader:  
        images, labels = images.to(device), labels.to(device)  
        outputs = ann_model(images)  
        _, predicted = torch.max(outputs, 1)  
  
        total += labels.size(0)  
        correct += (predicted == labels).sum().item()  
  
# 3.5.4 - Simpan accuracy untuk perbandingan nanti  
ann_accuracy = 100 * correct / total  
print(f"ANN Testing Accuracy: {ann_accuracy:.2f}%")  
  
... ANN Testing Accuracy: 96.02%
```

6. LANGKAH 6 - Bina model CNN

Contoh bentuk output:

```
Input batch: [64, 1, 28, 28]  
Selepas pooling: [64, ..., 14, 14]  
Output model: [64, 10]
```

```
cnn_model = CNN().to(device)  
print(cnn_model)  
  
# 3.6.4 - Loss function & optimizer  
criterion_cnn = nn.CrossEntropyLoss()  
optimizer_cnn = optim.Adam(cnn_model.parameters(), lr=0.001)  
  
... CNN(  
    (conv1): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
    (relu): ReLU()  
    (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)  
    (flatten): Flatten(start_dim=1, end_dim=-1)  
    (fc1): Linear(in_features=3136, out_features=128, bias=True)  
    (fc2): Linear(in_features=128, out_features=10, bias=True)  
)
```

7. LANGKAH 7 - Latih dan uji CNN

Contoh output semakan (nilai sebenar boleh berbeza):

```
Epoch 1/2 - Loss: 0.xxxx  
Epoch 2/2 - Loss: 0.xxxx  
CNN Testing Accuracy: xx.xx%
```

```
# 3.7.2 - Papar purata loss selepas setiap epoch  
avg_loss = running_loss / len(train_loader)  
print(f"Epoch {epoch+1}/{num_epochs} - Loss: {avg_loss:.4f}")
```

```
Epoch 1/2 - Loss: 0.2530  
Epoch 2/2 - Loss: 0.0786
```

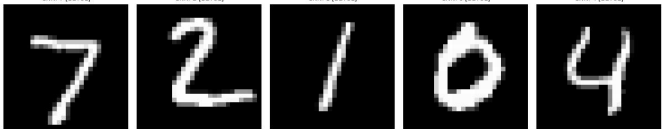
```
▶ # 3.7.3 - Tukar ke evaluation mode & kira accuracy  
cnn_model.eval()  
correct = 0  
total = 0  
  
with torch.no_grad():  
    for images, labels in test_loader:  
        images, labels = images.to(device), labels.to(device)  
        outputs = cnn_model(images)  
        _, predicted = torch.max(outputs, 1)  
  
        total += labels.size(0)  
        correct += (predicted == labels).sum().item()  
  
# 3.7.4 - Simpan accuracy untuk perbandingan  
cnn_accuracy = 100 * correct / total  
print(f"CNN Testing Accuracy: {cnn_accuracy:.2f}%")  
  
... CNN Testing Accuracy: 97.93%
```

8. LANGKAH 8 - Uji pengecaman lima imej digit

Contoh output bagi satu imej:
Actual: 7
ANN Prediction: 7 [BETUL]
CNN Prediction: 7 [BETUL]
[Paparan imej digit 7]

```
plt.tight_layout()
plt.show()

...
Actual: 7
ANN Prediction: 7 [BETUL]
CNN Prediction: 7 [BETUL]
...
Actual: 2
ANN Prediction: 2 [BETUL]
CNN Prediction: 2 [BETUL]
...
Actual: 1
ANN Prediction: 1 [BETUL]
CNN Prediction: 1 [BETUL]
...
Actual: 0
ANN Prediction: 0 [BETUL]
CNN Prediction: 0 [BETUL]
...
Actual: 4
ANN Prediction: 4 [BETUL]
CNN Prediction: 4 [BETUL]
...
```

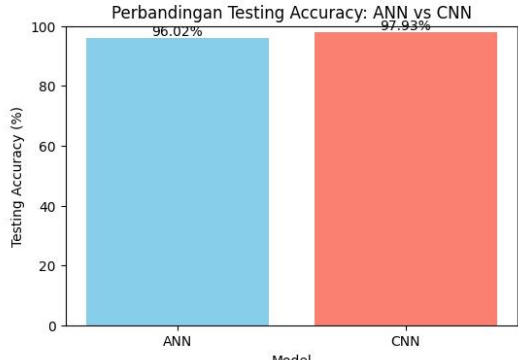


9. LANGKAH 9 - Bandingkan keputusan ANN dan CNN

MODEL	EPOCH	TESTING ACCURACY	RAMALAN BETUL / 5
ANN	2		
CNN	2		

```
else:
    print(f"Model dengan ramalan lebih tepat (5 imej): ANN dan CNN sama ({betul_ann}/5)")

...
=== JADUAL PERBANDINGAN ===
MODEL    EPOCH    TESTING ACCURACY    RAMALAN BETUL/5
ANN       2         96.02                5/5
CNN       2         97.93                5/5
```



Model dengan testing accuracy lebih tinggi: CNN (97.93%)
Model dengan ramalan lebih tepat (5 imej): ANN dan CNN sama (5/5)